

# Антология полезностей

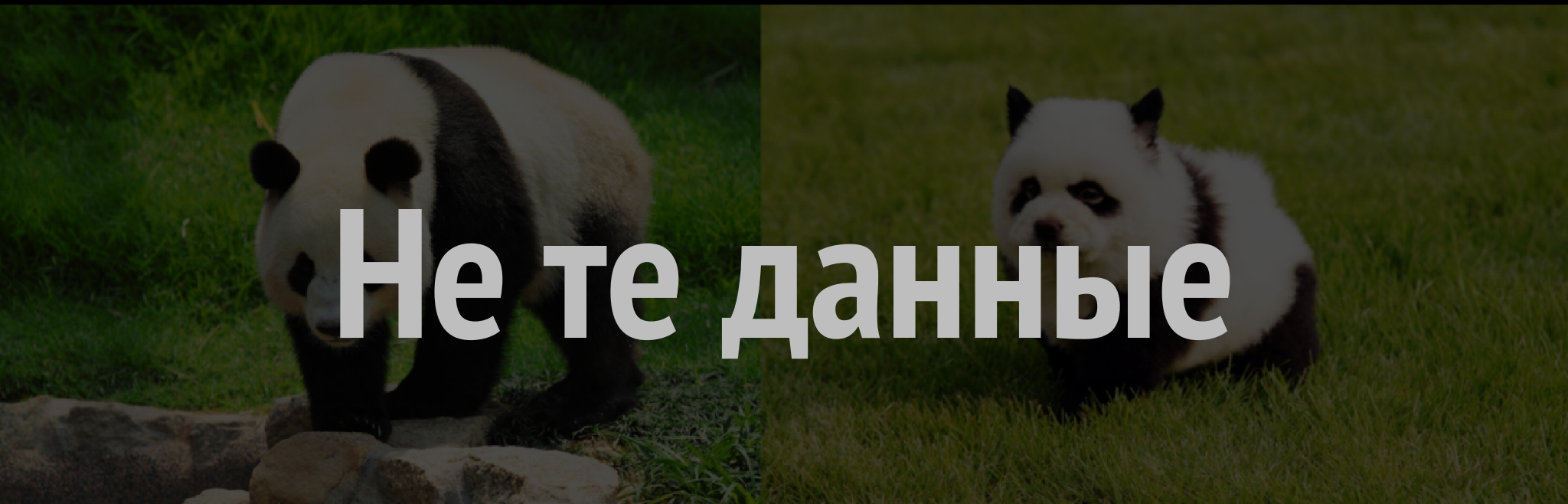
в TypeScript

# Андрей Тараненко

- Представляю холдинг T1
- 5 лет во frontend'e
- Все это время пишу на TypeScript
- Люблю типизировать



# Опрос / Agenda

The background image is a split-screen photograph of giant pandas. On the left, a large adult giant panda is shown in profile, standing on a rocky patch of ground and looking down. On the right, a small giant panda cub is sitting in a grassy field, looking towards the camera. The text 'Не те данные' is overlaid in the center in a large, white, sans-serif font.

Не те данные

# Работа с длинами и окружностями

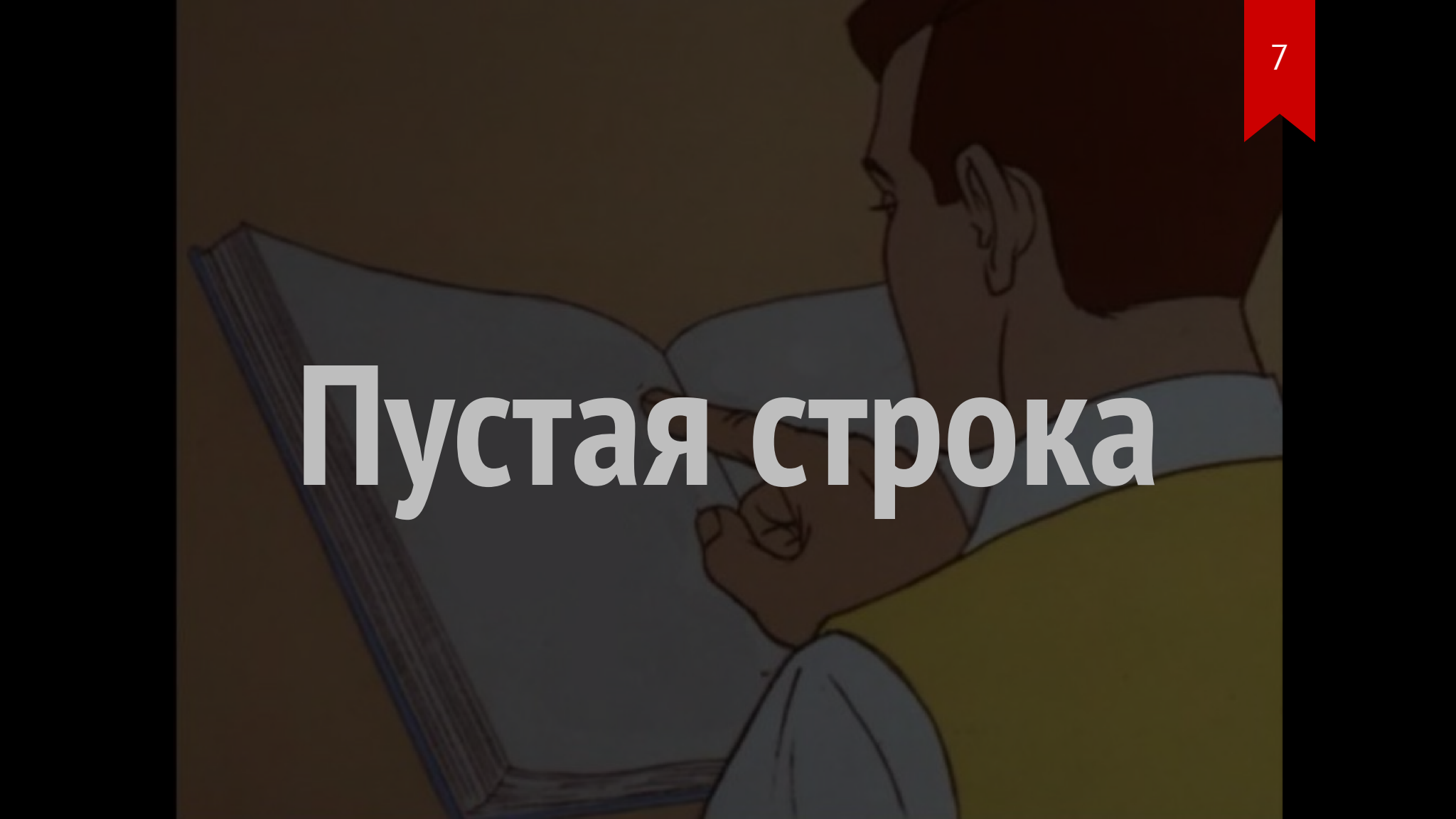
**Мы не одинаковые**

6

**Id разных сущностей**

**Мы разные**

# Пустая строка

A stylized illustration of a person with dark hair, seen from the side, reading a large open book. The person is wearing a light blue shirt and a yellow vest. The background is a dark brown gradient. The text 'Пустая строка' is overlaid in the center in a large, white, sans-serif font.

# Отправка метрик



# Не та нотация

**snake\_case в ДАННЫХ**

# Быстрый экскурс в основы

# Типы === Переменные

```
type HelloWorld = 'Hello' | 'World';
```

# Generic типы

# Generic – шаблон для создания типов

# Базовый синтаксис

```
type Generic<T> = {  
    payload: T;  
};
```

```
type StringPayload = Generic<string>;
```

**Generic === Функция**



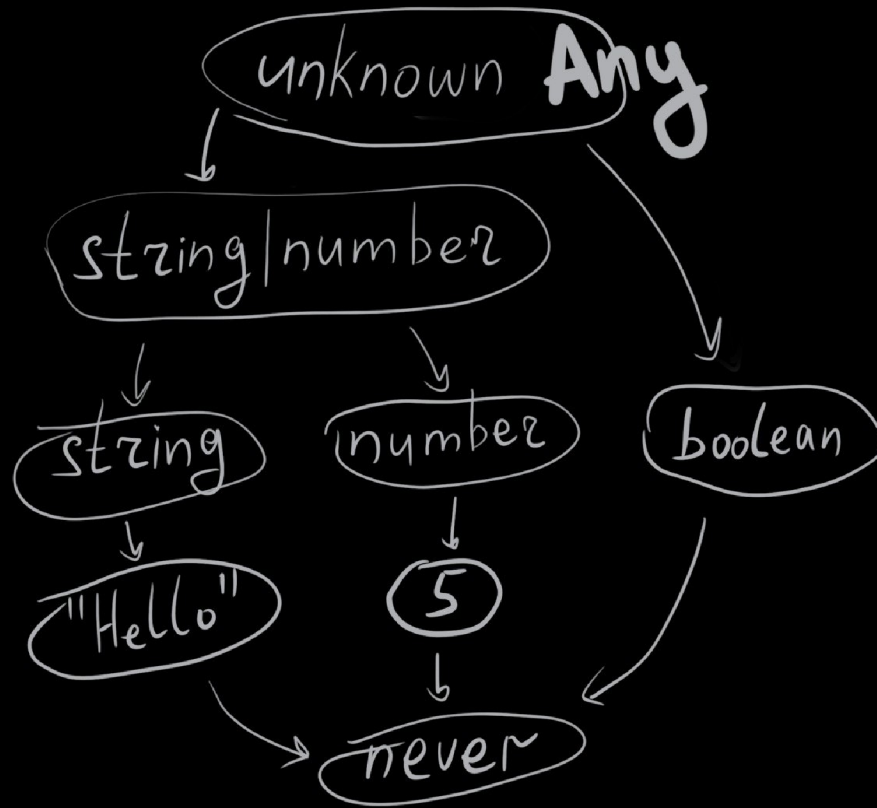
# Система типов TypeScript как язык программирования



**Frontend  
Conf 2024**

# Ограничение значений параметров

```
type Generic<T extends string> = /* */;
```



# Ограничение значений параметров

```
type Generic<  
    Key extends keyof T,  
    T extends string  
> = /* */;
```

# Ограничение значений параметров

```
type Generic<T extends Required<T>> = /* */;
```

# Условные типы

# Условные типы

```
type TArray<T> = T extends any[]  
  ? T  
  : T[];
```

```
TArray<string>; // string[]
```

```
TArray<string[]>; // string[]
```

# Вывод типов



# Вывод типов

```
type ArrayItem<T> = /* */;
```

**Можно ли что-то  
подставить, чтобы**

**условие было верным**

**Можно использовать  
для создания**

**перемешивающих**

# Создание переменных

```
type Generic<T> = GetNewType<T> extends infer U  
  ? U  
  : never;
```

# Template Literal Types

# Template Literal Types

```
type Name = 'world';
```

```
type Greet = `Hello ${Name}!`;
```

```
type Greet = 'Hello world!';
```

# Template Literal Types

```
type Name = 'world' | 'Everyone';
```

```
type Greet = `Hello ${Name}!`;
```

```
type Greet = 'Hello world!' | 'Hello Everyone!';
```

# Template Literal Types

```
type Greet = `Hello ${string}!`;
```



`\${string}`  
аналогичен (.+)

'true' | 'false'

`\${boolean}`

**Можно использовать  
как супертип**

# Можно использовать как супертип

```
type IsGreeting<T> = T extends `Hello ${string}!`  
  ? true  
  : false;
```

# Важное отличие от регулярок

**/^Шаблону  
сопоставляется**

**сразу вся строка\$ /**

# Шаблону сопоставляется сразу вся строка

```
type IsGreeting = 'Oh, Hello world!)))))' extends `Hello ${string}!`  
  ? true  
  : false
```

```
type IsGreeting = false
```

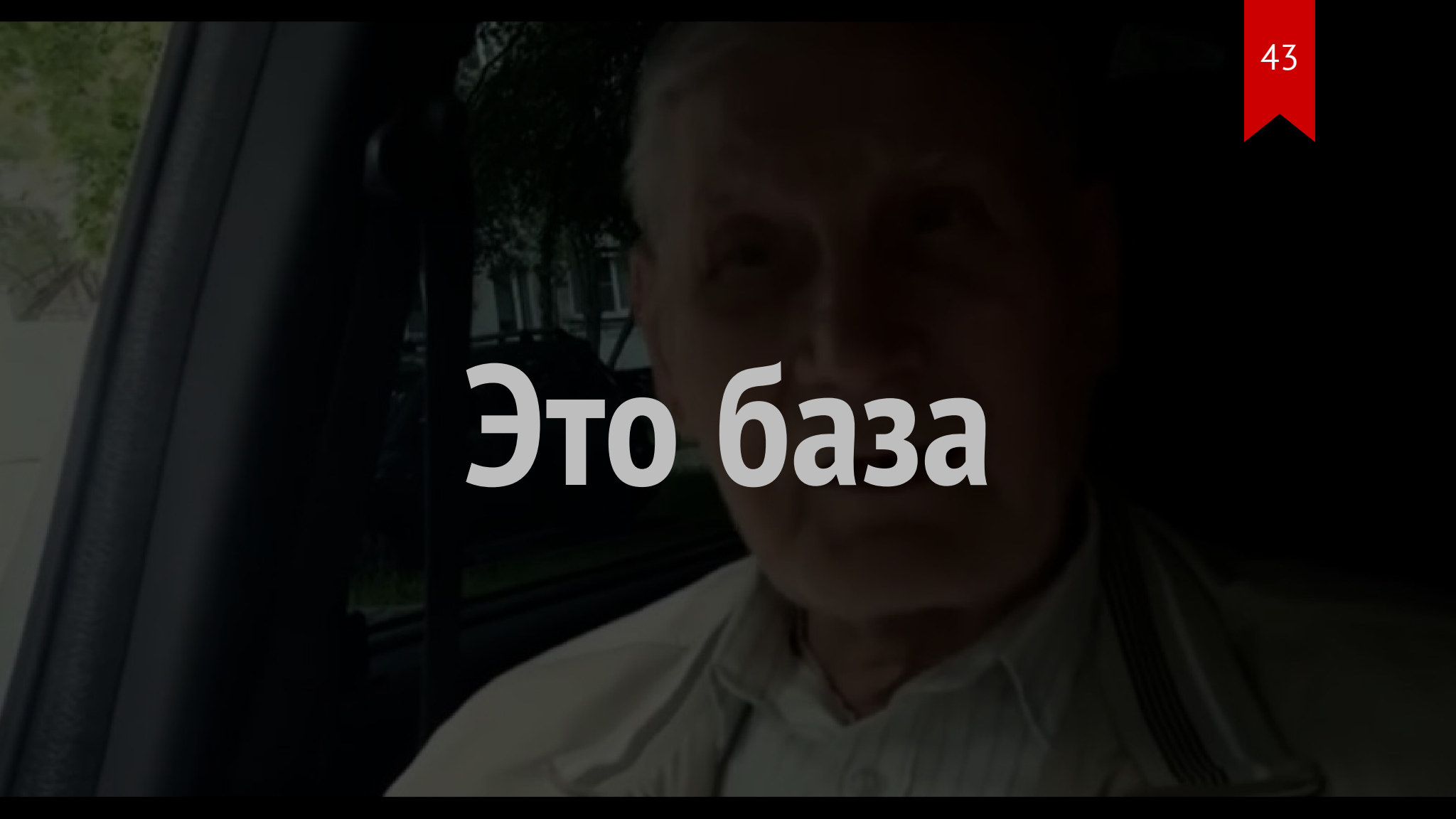


**Работает  
вместе с infer**

# Работает вместе с infer

```
type Greet = 'Hello world!' extends `Hello ${infer U}!`  
  ? U  
  : never;
```

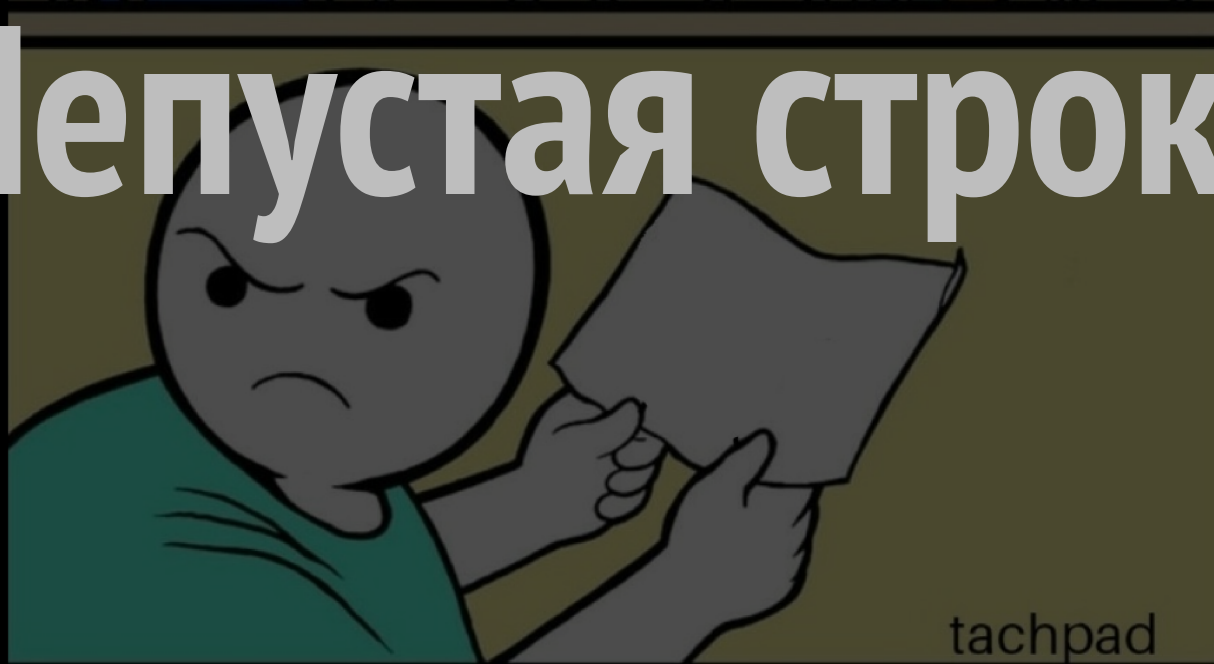
```
type Greet = 'world'
```



# Это база



# Непустая строка



# Непустая строка

```
const sendMetrics = (someImportantId: string) => {  
    /*    */  
};
```

```
sendMetrics(''); // 🤔
```

# Непустая строка

```
const sendMetrics = (someImportantId: string) => {  
    if (!someImportantId) { /* ?????????? */ }  
    /* */  
};  
  
sendMetrics(''); // 😬
```

# Как не дать передать пустую строку

`\${string}`  
аналогичен (.+)



# Непустая строка

```
type NotEmptyString = `${string}${string}`;
```

```
type NotEmptyString = string;
```

# Непустая строка

```
type NotEmptyString = `${any}${string}`;
```

# Непустая строка

```
type NotEmptyString = `${any}${string}`;  
  
const sendMetrics = (someImportantId: NotEmptyString) => {  
    /* */  
};
```

# Непустая строка

```
sendMetrics('1'); // Ok
```

```
sendMetrics('');
```

```
//      ^^ Несовместимые типы
```

# Непустая строка

```
const process = (someImportantId: string) => {  
  if (!someImportantId) return;  
  
  sendMetrics(someImportantId);  
  //          ^^^^^^^^^^^^^^^^^^^^^^ Несовместимые типы  
};
```

# Непустая строка

```
const process = (someImportantId: string) => {  
    if (!someImportantId) return;  
  
    sendMetrics(someImportantId as NotEmptyString);  
};
```

# Type Guard

```
const isEmptyString = (  
  value: string  
): value is NotEmptyString => !!value;
```

# Непустая строка

```
const process = (someImportantId: string) => {  
    if (!isEmptyString(someImportantId)) return;  
  
    sendMetrics(someImportantId);  
};
```

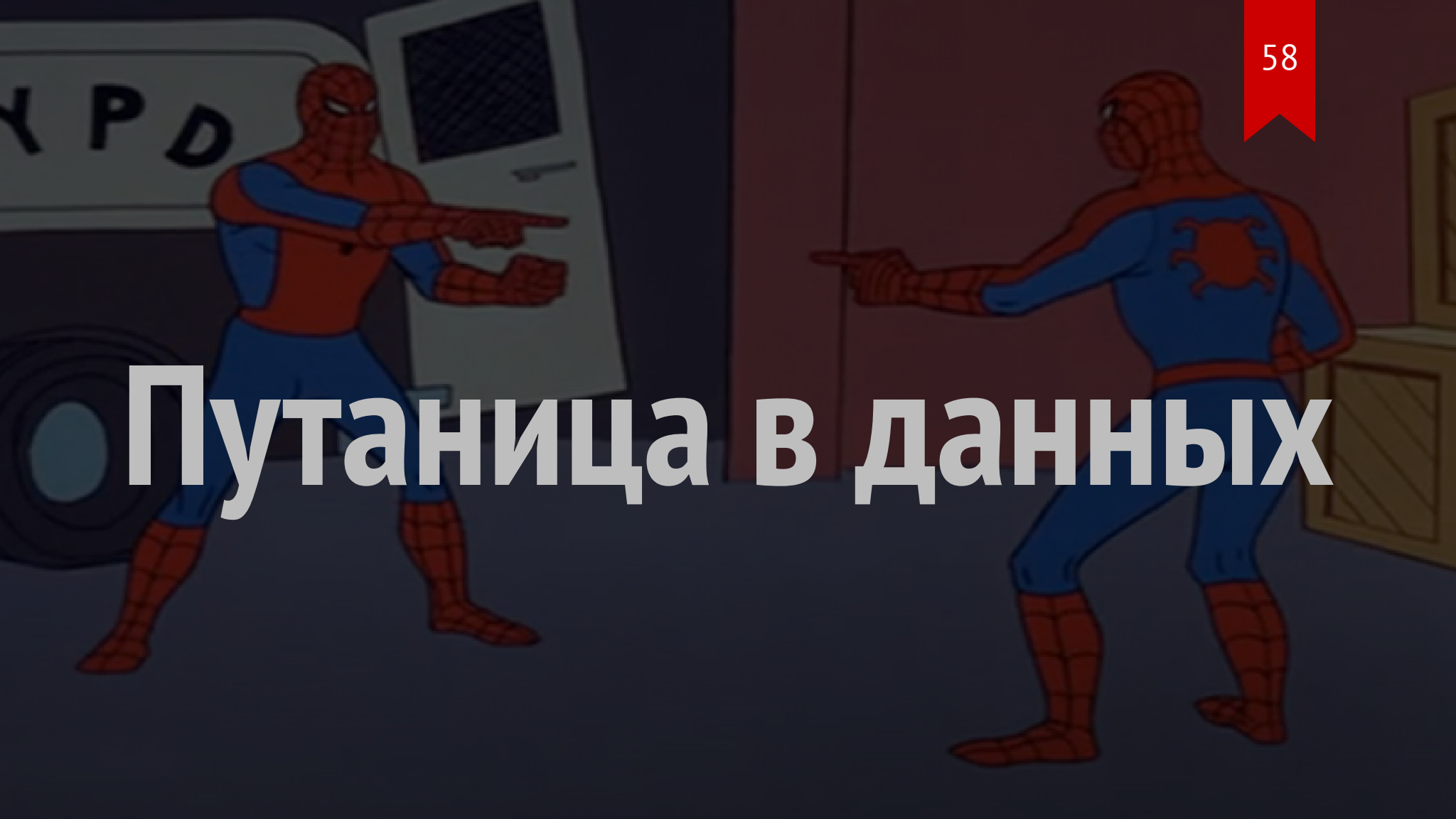


# Непустая строка

```
type AtLeastTwoChars = `${any}${string}${string}`;
```

```
type AtLeastThreeChars = `${any}${string}${string}${string}`;
```

# Путаница в данных



# Путаница в данных

```
const kilometersToMeters = (value: number) => value * 1000;  
const metersToKilometers = (value: number) => value / 1000;
```

# Путаница в данных

```
type Kilometers = number;
```

```
type Meters = number;
```

```
const kilometersToMeters = (value: Kilometers) => value * 1000;
```

```
const metersToKilometers = (value: Meters) => value / 1000;
```

# В TS структурная типизация

# Структурная типизация

```
type Kilometers = number;
```

```
type Meters = number;
```

```
Kilometers === Meters;
```

# Брендирование ТИПОВ

A full-page background image of Groot, the tree-like character from Guardians of the Galaxy. He is standing in a dark, reflective environment, possibly a wet street at night, with his right arm raised. The image is dark and moody, with Groot's brown, textured skin and green leaves being the primary light source.

Все  
есть  
объект



**Можно добавить  
уникальные поля**

# Можно добавить уникальные поля

```
type Kilometer = number & {  
  __brand: 'Kilometer';  
};
```

**unique** symbol

## unique symbol

```
const symbol = Symbol();
```

```
declare const symbol: unique symbol;
```

# Брендирование типов

```
declare const kilometersBrand: unique symbol;
```

```
type Kilometers = number & {  
  [kilometersBrand]: never;  
};
```

# Брендирование типов

```
type Brand<  
    TType,  
    TBrand extends symbol  
> = TBrand & { [key in TBrand]: never };
```

```
type BrandedNumber<  
    TBrand extends symbol  
> = Brand<number, TBrand>;
```

# Брендирование типов

```
type Kilometers = BrandedNumber<typeof kilometersBrand>;
```

```
type Meters = BrandedNumber<typeof metersBrand>;
```

```
const kilometersToMeters = (value: Kilometers) => value * 1000;
```

```
const metersToKilometers = (value: Meters) => value / 1000;
```

# Брендирование типов

```
let distance!: Kilometers;  
kilometersToMeters(distance);  
metersToKilometers(distance);  
//                ^^^^^^^^ Не совместимые типы
```



# Брендирование типов

```
metersToKilometers(1000);
```

```
metersToKilometers(1000 as Meters);
```

# Брендирование типов

```
type Meters = BrandedNumber<typeof metersBrand>;  
type Radius = BrandedNumber<typeof radiusBrand>;  
  
const circleSquare = (radius: Radius & Meters) =>  
    Math.PI * radius ** 2;
```

**snake\_case в ДАННЫХ**

## snake\_case в данных

```
type ResponseData = {  
    time_stamp: string;  
    short_name: string;  
    full_name: string;  
};
```

# Дублирование

```
type ResponseData = {  
    time_stamp: string;  
    short_name: string;  
    full_name: string;  
};
```

```
type Data = {  
    timeStamp: string;  
    shortName: string;  
    fullName: string;  
};
```

**Нужно следить  
за изменениями**

**в 2х местах**

**Может стоит  
преобразовать тип?**

# Object To Camel Case



# План

- Преобразование объекта
- Преобразование имен ключей

# Преобразование объекта

# Mapped types

# Mapped types

```
type Type = { /* ... */};
```

```
type MappedType = {  
    [Key in keyof Type]: Type[Key];  
}
```

# Mapped types

```
type Type = { /* ... */};
```

```
type MappedType = {  
  readonly [Key in keyof Type]?: Type[Key];  
}
```

# Mapped types

```
type Type = { /* ... */};
```

```
type MappedType = {  
  +readonly [Key in keyof Type]: Type[Key];  
}
```

# Mapped types

```
type Type = { /* ... */};
```

```
type MappedType = {  
  [-readonly [Key in keyof Type][-?: Type[Key];  
}
```

# Mapped types

```
type Type = { /* ... */};
```

```
type MappedType = {  
  [Key in keyof Type] as Transform<Key>[]: Type[Key];  
}
```



**Можно  
отфильтровать**

**объект**

# Преобразование имен ключей

# Snake To Camel Case

# Алгоритм

1. Найти \_
2. Правую часть капитализировать
3. Повторить алгоритм для правой части
4. Объединить с левой частью

# Алгоритм

```
SnakeToCamelCase<'snake_case_string'>
```

```
'snake' 'case_string'
```

```
'snake' 'case_string'
```

```
'snake' 'Case_string'
```

```
'snake' + SnakeToCamelCase<'Case_string'>
```

# Snake To Camel Case

```
type FromSnakeToCamelCase<
    SnakeCaseString extends string,
    ProcessedPart extends string = ''
> = SnakeCaseString extends `${infer Word}_${infer Rest}`
    ? FromSnakeToCamelCase<Capitalize<Rest>, `${ProcessedPart}${Word}`>
    : `${ProcessedPart}${SnakeCaseString}`;
```

A зачем  
ProcessedPart?

## Без ProcessedPart

```
type FromSnakeToCamelCase<
    SnakeCaseString extends string
> = SnakeCaseString extends `${infer Word}_${infer Rest}`
    ? `${Capitalize<Word>}${FromSnakeToCamelCase<Rest>}`
    : SnakeCaseString;
```





# Оптимизация хвостовой рекурсии

# Без дублирования

```
type ResponseData = {  
    time_stamp: string;  
    short_name: string;  
    full_name: string;  
};  
  
type Data = ObjectToCamelCase<ResponseData>
```

# Комбинированные типы

```
type ShortData = { id: string };
```

```
type AdditionData = { value: string };
```

```
type Data = ShortData & AdditionData;
```

**Хочется красиво**

# Prettify

```
type Prettify<T> = T extends object
  ? { [K in keyof T]: Prettify<T[K]> }
  : T;
```

# UserInfo

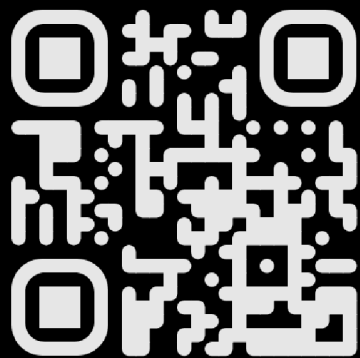
```
type Data = Prettify<ShortData & AdditionData>;
```

```
type Data = {  
  id: string;  
  value: string;  
};
```

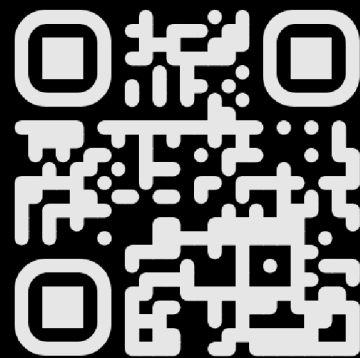
# Ссылка на слайды



# Спасибо за внимание



[@i\\_oktav\\_i](#)



[@typescript\\_bowl](#)